
Playing Through an AI Incident

Sofia G. Gallego
Cosmoventures, Paris, France

Abstract

We present *Little Agent Lab*, a free, browser-based, deterministic puzzle game built for the communication track of the Apart AI Incident Response Sprint. The game teaches three related oversight concepts — capability separation, gated review, and the fact that a labeled safeguard may not actually function — through three escalating missions built around a single, always-visible goal statement. Unlike a live model, every “agent” is a small scripted state machine, so behavior is fully reproducible and safe to inspect. We report the design rationale, three major iterations driven by direct usability feedback, and our verification method: 27 automated logic tests plus headless-browser screenshot review of every mission and outcome path. We did not run a formal study with naive users, and we report this omission plainly rather than implying validation we do not have. We argue the project’s contribution is a concrete, replicable communication pattern — “verify what a safeguard does, not what it’s called” — expressed as playable mechanics rather than prose, and we describe what a follow-up evaluation would need to measure.

***Epistemic status:** This is a solo, weekend-timeframe project report. Claims about design rationale and what was tested are accurate to the best of the author’s knowledge and are checked against the actual codebase and automated test/screenshot logs referenced below. Claims about pedagogical effectiveness are explicitly not validated — no human-subject study was run — and are flagged as such throughout rather than implied by omission.*

1 Introduction

Most public-facing explanations of AI oversight and permissioning are prose: blog posts, explainer diagrams, or abstract descriptions of “least privilege” and “monitoring.” These are accurate but inert — a reader can nod along without ever having to notice that a check has failed. This project addresses a narrower sub-problem within AI safety communication: **how do you get a non-expert (including a child) to personally experience, rather than just read about, the difference between a system that is safe because it was built with separated capabilities and a system that merely looks safe because a checkpoint has a reassuring name?**

This sits in the *communication / public understanding* region of the sprint’s problem space rather than in technical red-teaming or evaluation. The theory of change is indirect: better public and early-education intuitions about oversight and verification (as opposed to blind trust in a label like “inspector” or “moderated”) are a small but real input into how the next generation of users, policymakers, and junior engineers reason about AI systems they did not build themselves. This is not a claim that the game measurably changes real-world behavior — we have not tested that — only that it is a small, inspectable artifact aimed at that gap.

The concrete deliverable is a static, client-side web game (no accounts, no network calls, no live model in the loop) in which the player assembles a small team of fictional “agents” to complete a task, while keeping a private item from leaving a workshop unless it has actually been checked. Three missions

escalate the difficulty of that judgment call.

2 Related Work

Security- and safety-awareness games are an established genre outside AI safety specifically. *Anti-Phishing Phil* [5] used an embedded game to teach URL-inspection habits and reported measurable gains in phishing-detection accuracy in a lab study — the closest methodological precedent for what a real evaluation of this project would need to look like (pre/post detection-accuracy testing against matched control content). *Control-Alt-Hack* [3] used a card game to teach computer-security concepts to non-specialists, including the idea that a system can look secure while a specific component silently fails. Our “lookalike inspector” mission (Section 3.3) is a direct, much smaller-scope descendant of that idea, restated for an AI-oversight framing.

Within AI safety specifically, the “specification gaming” list maintained by Krakovna et al. [4] documents real cases where a system satisfied the letter of a check while violating its intent — the phenomenon our decoy-inspector mission dramatizes in miniature and entirely fictionally. Concrete-Problems-style framings of oversight and scalable supervision [1] motivate why “a checkpoint exists” is not the same claim as “a checkpoint works,” which is the single idea this game is built around.

We are not aware of an existing browser game that isolates *this specific* distinction (a nominally-present safeguard that does not actually block anything) as its central, escalating mechanic for a general/lay audience. We make no claim of novelty in the general genre of “security awareness games” — only in this narrow framing and in bundling it with capability-separation and gated-review mechanics inside one small, testable, dependency-free artifact.

3 Methodology

3.1 Affordance level and controls

Everything in the game is deterministic and fully scripted in TypeScript; there is no live model, no external API call, and no randomness beyond which of a small fixed set of pre-authored branches a given team selection triggers. This was a deliberate control: the game’s job is to communicate a concept clearly, not to demonstrate an actual model being unreliable, which would require far more infrastructure and would risk misrepresenting how real model behavior works. Every “agent” (e.g. a builder, a fetcher, an inspector) is a named function branch, not a language model.

3.2 Iteration history

The project went through three materially different designs within the sprint window, each replaced after concrete, specific usability failures were identified — not through a formal study, but through direct, adversarial-style feedback from a single evaluator (the requester in this solo project) treated as a stand-in end user. We report this honestly as an $n=1$, non-blinded, non-representative process, not a usability study.

Version	Design	Failure identified
v1	Single fixed scenario (“Weather Lantern”): pick a team, run once, read the outcome.	Once solved, no replay value; also the initial “obvious tell” in decision text gave away the correct answer before the mechanic mattered.
v2	Abstract four-level “ticket triage” game: a stream of decontextualized requests to Allow/Block, escalating in concurrency and time pressure.	A real implementation bug (UI never re-rendered after populating the request queue) produced an apparently frozen screen; even after that fix, requests were shown as bare code with no plain-language explanation, and a mandatory multi-step tutorial blocked entry. Rejected by the evaluator as confusing and not “game-like.”
v3 (final)	Returned to the v1 team-builder mechanic, added a persistent, unmissable goal callout, an optional (dismissible) “How to play” panel instead of a forced tutorial, and three escalating missions unlocked in sequence, each with a distinct required strategy.	Accepted. Verified by automated tests and headless-browser screenshot review (Section 3.4) rather than by the evaluator’s manual inspection alone.

Table 1. Summary of the three design iterations within the sprint window.

We flag this history explicitly because it is the most defensible evidence in this report: not “the design is good” (unverified), but “a specific, more novel design was tried, produced concrete usability failures, and was abandoned in favor of a simpler mechanic with narrower but real content variation.”

3.3 Final design

The final game presents one persistent objective panel per mission, stating both win conditions as numbered steps (“get the private item combined with the public information and finish the job” and “never let the private item leave the workshop unchecked”) rather than as prose. The player selects one “builder” card and up to two “helper” cards, sets a gate toggle, and runs the plan; the result plays out as a timed, line-by-line action trace using literal code-like statements (e.g. `lens.deny(payload.private === true)`) so that what actually happened is inspectable and not just narrated.

Three missions each isolate one lesson:

1. **Weather Lantern.** Two independently valid solutions exist: use a helper whose route structurally never touches the private item, or open the gate but pair it with an inspector that actually checks outgoing requests. Teaches capability separation as one valid strategy among others.
2. **Garden Delivery.** No safe “structurally separated” helper exists in this mission. The cautious builder character cannot win under any configuration; the only winning combination requires deliberately taking the outside route while pairing it with the inspector. Teaches that caution alone is not always a viable strategy — sometimes engaging with risk, correctly checked, is the only path to completion.
3. **Message Relay.** Two inspector-labeled helper cards are offered (“Warden” and “Rusty”); only one actually blocks anything, and the card text for both is deliberately non-revealing (“what it actually does, you’ll only know once you run it”). This is the game’s direct expression of “don’t trust the label” — the player must run the plan and read the action trace to find out which safeguard is real.

Each mission’s simulation is a small, hand-written branching function, and each branch — including every win and every loss path — is covered by an automated test asserting the intended outcome (Section 3.4).

3.4 Verification method

In the absence of a human-subject study, we verified two separable claims: (a) that the simulation logic matches its intended design for every reachable combination, and (b) that the interface actually renders what the logic produces, with no runtime errors, across all three missions and both win/lose outcomes.

For (a), we wrote 27 automated unit tests (Node’s built-in test runner) that construct every meaningfully distinct team configuration per mission and assert the resulting `complete/protected` flags and the presence of specific event kinds (e.g. that a `leak` event occurs exactly when expected). All 27 pass. For (b), we drove the built app with headless Chromium (Playwright), navigating through the goal panel, a full mission win, a mission-unlock transition, and a second mission’s distinct card set, capturing screenshots and console-error logs at each step. No console errors were observed in any captured state.

We are explicit that this verifies *internal correctness and non-brokenness*, not pedagogical effectiveness. A ten-year-old encountering “Warden” and “Rusty” for the first time may or may not form the intended lesson; we did not test that.

4 Results

Because this is a communication-track artifact rather than a model-evaluation project, we do not have activation-rate, selectivity, or false-positive-rate metrics in the interpretability sense. The results we can report are:

- **Logic correctness:** 27/27 automated tests passing across all three missions, covering every intended win path, every intended loss path, and — for Mission 3 specifically — all four combinations of the two inspector cards (real only, fake only, both, neither), confirming the decoy mechanic behaves exactly as designed in every combination, not just the two “obvious” ones.
- **Interface correctness:** Zero console errors across five distinct captured application states (initial load, goal panel, a completed first mission, a mission-unlock transition, and a second mission’s distinct card set), verified via headless-browser screenshots rather than by description.
- **Iteration signal:** Two of three design generations were discarded after concrete, specific usability failures (a broken re-render; an unexplained abstraction paired with a forced tutorial). We treat the fact that specific, falsifiable problems were identified and fixed — rather than vague dissatisfaction — as the main process-level result of this report.
- **What we did not measure:** comprehension before/after play, engagement/completion rate among naive players, transfer of the “verify, don’t trust the label” heuristic to any non-game context, or accessibility (screen-reader, keyboard-only, or child-readability testing beyond the author’s own inspection).

5 Discussion

The central design bet of this project is that a single, narrow, well-isolated mechanic — a labeled safeguard that does not do what its label implies — is more likely to leave a lasting intuition than a longer, more realistic simulation would, because the player experiences the failure of their own trust directly rather than being told about it. This is consistent with the security-education literature cited in Section 2, but we have not tested whether it holds for this specific game, this specific age range, or this specific framing.

A second, less deliberate finding worth reporting: across three iterations, the version that generated the strongest negative reaction was not the simplest one (v1) but the most mechanically novel one (v2, the abstract ticket-triage game). Novelty and clarity pulled in opposite directions in this case; the eventual design (v3) kept v1’s legible mechanic and added content variation (three missions) rather than

mechanical complexity. We think this is a useful, if anecdotal, data point for anyone building communication artifacts under sprint time constraints: a familiar, well-explained mechanic with new content may communicate a concept more reliably than a novel mechanic that needs its own tutorial.

Future work that would materially strengthen this report: (i) a small pre-registered comprehension study — a short pre/post question such as “if a safeguard is present, is a request automatically safe?” — with a matched control group who read an equivalent-length prose explanation instead of playing; (ii) a completion-rate and time-on-task study with an age-appropriate sample (the game’s own framing targets ages 8+); (iii) accessibility testing (screen reader, keyboard-only, color-contrast) which has not been done; (iv) extending the decoy-safeguard mechanic to a fourth mission where the player must design or select which check to trust from a larger pool, to test whether the lesson generalizes past the two-card case tested here.

Limitations and Dual-Use Considerations

Limitations

- **No human-subject evaluation.** Every claim about the game being clear, engaging, or pedagogically effective is untested against real users; all usability iteration in Section 3.2 came from a single non-blinded evaluator, which is a weak and potentially unrepresentative signal.
- **Small scenario space.** Three missions is enough to demonstrate the mechanic, not enough to establish that the lesson generalizes past this specific game’s framing.
- **Oversimplification risk.** Real AI safeguards are frequently far less legible than “run the plan and read a five-line trace” — a player could walk away with false confidence that verifying a real system’s safeguards is similarly fast or simple. We do not currently mitigate this beyond a closing note in the in-game “what does this model simplify” panel, which itself has not been tested for whether players read or absorb it.
- **Single-developer bias.** All design, implementation, and informal evaluation were done by one person in a compressed timeframe, with AI-assisted implementation (Claude Code) used throughout for coding and drafting; this report was reviewed and directed by the author but drafted with substantial AI assistance, which the author discloses per the sprint’s own guidance on accurately describing contributions.
- **No accessibility testing.** Screen-reader, keyboard-only, and child-readability testing were not performed beyond the author’s own manual inspection of ARIA roles and focus states in the markup.

Dual-Use Considerations

We assess the dual-use risk of this project as low. The game teaches, in an entirely fictional setting with invented agent names and invented destination strings, that a nominally protective check can fail to function — it does not teach any technique applicable to a real system: there is no real API surface, no real destination format, no real model, and no content generated by or extracted from a live model at any point. The mechanic that most closely resembles a “bypass” (Mission 3’s decoy inspector) is a fixed, pre-authored branch selectable from two labeled cards, not a generative or discoverable exploit; a player cannot use anything learned in this game to construct a real bypass against a real system, because no real system’s actual naming, protocol, or verification logic is represented. We did not use any model to generate harmful content, jailbreak any model, or interact with any organization’s real systems in building this project, consistent with the sprint’s stated constraints.

References

- [1] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete Problems in AI Safety. *arXiv:1606.06565*.

-
- [2] Apart Research. AI Incident Response Sprint (Sept 11–13, 2026). <https://apartresearch.com/sprints/ai-incident-response-sprint-2026-09-11-to-2026-09-13>.
 - [3] Denning, T., Lerner, A., Shostack, A., & Kohno, T. (2013). Control-Alt-Hack: A Card Game for Computer Security Awareness and Education. *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (CCS '13)*.
 - [4] Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., & Legg, S. (2020). Specification gaming: the flip side of AI ingenuity. *DeepMind blog*.
 - [5] Sheng, S., Magnien, B., Kumaraguru, P., Acquisti, A., Cranor, L.F., Hong, J., & Nunge, E. (2007). Anti-Phishing Phil: The Design and Evaluation of a Game That Teaches People Not to Fall for Phish. *Proceedings of the 3rd Symposium on Usable Privacy and Security (SOUPS '07)*.

Project source: <https://github.com/nefinia/little-agent-lab> (public; playable build in the repository's GitHub Pages workflow). No installation recipe or exploit content is included, so no disclosure review was needed beyond this report's own dual-use assessment above.